TELECOM Nancy 2A
Graphes et Recherche Opérationnelle 1

TD machine 2 : résolution de problèmes de sac à dos

1 Introduction

On considère le problème de sac à dos $(\mathcal{P})$ suivant : trouver un vecteur $x \in \{0,1\}^n$ qui maximise une fonction :

$$f(x) = c^\top x = c_0 x_0 + \dots c_{n-1} x_{n-1}$$

sous les m contraintes (écrites matriciellement) $Ax \leq b$, où :

$$c_j > 0, b_j > 0, \forall j \in [\![0, n-1]\!] \text{ et } a_{i,j} \geq 0, \forall (i,j) \in [\![0, m-1]\!] \times [\![0, n-1]\!]$$

mais aucune colonne ne doit être nulle ($\forall j, \exists i : a_{i,j} > 0$). Un tel problème est défini uniquement par la donnée de c, A et b. Dans ce TP vous allez :

- coder des heuristiques permettant de déterminer une solution réalisable de $(\mathcal{P})$;
- coder la méthode PSE naïve vu en CM et en TD pour résoudre $(\mathcal{P})$ (en intégrant la meilleure heuristique trouvée pour le problème donné à résoudre) ;
- et enfin résoudre $(\mathcal{P})$ en utilisant le solveur gurobi.

Une partie du code est déjà écrite et doit être récupérée sur Arche, vous disposez de :

- `util_tp2.py` : ce module contient le type `TreeNodePSE` permettant de créer les noeuds de l'arbre d'exploration ainsi que des fonctions fournissant les données c, A, b pour différents problèmes de sac à dos. **Ne pas modifier ce fichier.**
- `heuristiques_squelette.py` contenant une fonction très pratique pour appliquer une heuristique et les entêtes des fonctions qu'il faudra ajouter dans ce module. **Renommer ce fichier** `heuristiques.py`.
- `solveur_pse_squelette.py` contenant une petite partie du code. **Idem penser à renommer ce fichier** `solveur_pse.py`.

2 Heuristiques

Vous allez utiliser les heuristiques suivantes :

$\mathcal{H}_1$: on examine les variables selon l'ordre décroissant des coefficients c de la fonction objectif.

$\mathcal{H}_{2,X}$: on examine les variables selon l'ordre décroissant des rapports "utilité sur consommation de contrainte", soit $r_j := \frac{c_j}{a_j}$ lorsqu'il n'y a qu'une seule contrainte, et sinon selon (cf ex 2, td 3) :

$$r_j := \frac{c_j}{\|p^j\|_X} \text{ où } p^j = (\frac{a_{0,j}}{b_0}, \dots, \frac{a_{m-1,j}}{b_{m-1}})$$

avec par exemple $X = 1$ et $X = \infty$ (soit 2 heuristiques supplémentaires ou plus si vous voulez essayer d'autres normes comme la norme 2).

Travail à réaliser

- Écrire (compléter à partir des entêtes dans le fichier `heuristiques.py`) le code correspondant aux différentes heuristiques. Pour cela, une fois que vous avez établi les quantités qu'il faut trier, il suffit d'appeler la méthode `argsort` qui retourne un vecteur d'indices (donnant l'ordre du tri croissant) puis d'appeler la fonction `applique_heuristique` déjà prête qui calcule la solution réalisable selon l'ordre des variables à examiner (rmq : cette fonction tient compte du fait que `argsort` trie dans l'ordre croissant). Aide (trucs numpy) : comme les données c, A, b sont des tableaux numpy, le vecteur p^j précédent peut s'obtenir comme la colonne j du tableau :

```
P = A/b.reshape((m,1))    # broadcasting non élémentaire....
```

Et il est possible de calculer la norme de chaque colonne de `P` avec :

```
norm_P = np.linalg.norm(P,axis=0, ord=1)  # axis=0 pour calculer la norme de chaque colonne
                                          # ord=np.inf pour la norme infinie,
                                          # ord=2 pour la norme 2, etc.
```

et les rapports r peuvent alors se calculer avec `r = c/norm_P`. Remarque : dans le cas d'une seule contrainte ($m = 1$), toutes les heuristiques $h_{2,X}$ sont identiques.

— Test des heuristiques. Il y a 9 problèmes de sac à dos disponibles dans `util_tp2.py`. Les données pour le problème x ($x = 1, 2, \ldots, 9$), s'obtiennent avec l'appel `c, A, b = pbsac`x`()`. Voici ce que vous devez obtenir pour ces heuristiques sur les 9 problèmes (m nb de contraintes, n nombre de variables) :

	m	n	$\bar{F}_1$	$\bar{F}_{2,1}$	$\bar{F}_{2,\infty}$	$\bar{F}_{2,2}$
*pbsac*1	2	4	**9**	8	8	8
*pbsac*2	1	20	**1025**	1019	1019	1019
*pbsac*3	10	15	**3950**	3825	3760	3825
*pbsac*4	6	18	129	153	**162**	155
*pbsac*5	10	20	**6070**	5455	5530	5455
*pbsac*6	5	39	9209	9888	**10295**	10148
*pbsac*7	10	28	**12180**	11970	11970	11970
*pbsac*8	5	50	13863	15540	15793	**15832**
*pbsac*9	1	100	**979**	965	965	965

— Écrire (compléter) la fonction qui lance toutes les heuristiques et permet de choisir la meilleure pour le problème donné (la fonction doit retourner $\overline{x}$ et $\overline{F} = f(\overline{x})$).

3 La méthode PSE

Travail à réaliser

Le but est de compléter le squelette fourni pour la fonction `solveur_pse_bin` puis de le tester (et sans doute déboguer) sur plusieurs exemples. Auparavant il faut :

— relire la partie du cours correspondante (au cas où vous l'auriez oubliée) pages 41 à 43 ;

— lire les compléments d'information exposés juste ci-après et bien lire et comprendre le squelette fourni.

Compléments d'information :

— La méthode PSE naïve est consommatrice de mémoire. Ainsi chaque noeud de l'arbre ne comporte que l'information minimale nécessaire à savoir :

  — le noeud parent (`None` pour le noeud racine de l'arbre d'exploration) ;

  — la potentialité [1] F_{max} de la fonction coût du noeud (c'est à dire la valeur de f si toute les variables dont la valeur n'a pas encore été fixée seraient égale à 1) ;

  — ce qu'il reste comme provision "à manger" pour le second membre des contraintes [2] ;

  — le numéro de la variable correspondante $0, 1, \ldots, n-1$ (-1 pour le noeud racine) ;

  — la valeur (0 ou 1) de cette variable.

1. Dans le cours, pour un noeud S_k de l'arbre ce terme F_{max} est appelé b_k.

2. Ce terme pour un sommet S_k est appelé e_k dans le cours mais il peut être vectoriel si on a plus d'une contrainte ; dans cet énoncé on l'appelle `Cstr`.

Un noeud ne comporte donc pas de références sur ses enfants (2 enfants maximum) mais uniquement sur son parent, ce qui est nécessaire pour reconstituer la solution lorsqu'on arrive à un noeud terminal (i.e. noeud dont le numéro de la variable est $n-1$). Pour créer un noeud, il faut utiliser la classe (et la fonction) `TreeNodePSE` :

```
N = TreeNodePSE(parent, Fmax, Cstr, num_var, val_var)
```

— Les noeuds (déjà créés) de l'arbre sont examinés selon la potentialité maximale. Il s'agit donc de les mettre dans une file de priorité qui est implémentée en général via un tas. En python, un tas est obtenu avec une simple liste aidée des fonctions suivantes :

```
from heapq import heappush, heappop
```

Si `T` est la liste "servant" de tas, alors :
- on initialisera le tas par la liste vide `T = []` ;
- `heappush(T,N)` pour mettre un noeud N nouvellement créé dans la file de priorité T. **Important :** il n'y a pas de priorité à rajouter car la classe `TreeNodePSE` implémente déjà l'opérateur de comparaison adéquat.
- `N = heappop(T)` pour extraire de la file de priorité le noeud qui a la potentialité la plus grande (le F_{max} le plus grand).
- pour tester que le tas est non vide, on peut utiliser `len(T) > 0`.

— Lorsque un noeud N est sorti de la file, on se pose plusieurs questions :
- Est-ce que son F_{max} est inférieur ou égal au $\overline{F}$ courant ? Si oui on arrête l'exploration (ceci est déjà codé dans le squelette).
- Peut-on créer un enfant gauche (celui pour lequel on mettrait à 1 la variable x_j) ? Rmq : pour obtenir j le numéro/indice de la variable, il faut ajouter 1 à celui de son parent !
- Peut-on créer un enfant droit (l'autre avec $x_j = 0$) ?

— Une fois un nouveau noeud créé (gauche ou droit), il faut voir s'il s'agit d'un noeud terminal ou pas (dans ce dernier cas un tel noeud est mis dans la file de priorité). S'il est un noeud terminal ($j = n-1$) alors c'est qu'il permet d'augmenter le $\overline{F}$ courant et il faut mettre jour les variables `Fb` et `xb`. Pour mettre à jour ce dernier (le noeud terminal ne donne que la composante $n-1$), il faut donc remonter l'arbre : le parent du noeud terminal donne la composante $n-2$, son grand-parent la composante $n-3$, etc...

— la fonction `solveur_pse_bin(c, A, b, xh, Fh)` doit retourner 3 arguments, la solution optimale x^* (contenue dans la variable `xb`), $f(x^*)$ (contenue dans la variable `Fb`) ainsi que le nombre de noeud créés (variable `nb_node`).

Tests

test 1 Commencez par les données c, A, et b fournies par la fonction `pbsac1`, c'est à dire celles de l'exercice 2 de la feuille 5, comportant 4 variables et 2 contraintes dont la solution est $x^* = (1, 0, 0, 1)$ et $F^* = 10$. Il y a 10 noeuds créés (9 si vous avez codé sans créer de noeud terminal). Si vous rencontrez des difficultés, plutôt que de faire des `print` intempestifs, essayez le debogueur (cf feuille distribuée en plus de cet énoncé).

test 2 Utilisez les données de `pbsac2` (20 variables, 1 contrainte). Pour ce problème la meilleure heuristique est une solution exacte ($F^* = 1025$). 186 noeuds sont crées.

test 3 Données de `pbsac3` (15 variables, 10 contraintes). La meilleure heuristique donne $\overline{F} = 3950$, la solution optimale obtenue est $x^* = (1, 1, 0, 1, 0, 1, 1, 0, 1, 1, 0, 0, 0, 1, 1)$ avec $F^* = 4015$. 3670 noeuds ont été créés.

test 4 Pour `pbsac4` (18 variables, 6 contraintes). La meilleure heuristique donne $\overline{F} = 162$, la solution optimale obtenue est $x^* = (0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1)$ avec $F^* = 167$. 3498 noeuds ont été créés (3496 si vous ne créez pas de noeud terminal). Vous pouvez mesurer le temps de cette façon :

```
from time import time
......
t0 = time()
# code dont on veut mesurer le temps
tc = time() - t0
```

Pour cet exemple j'obtiens 0.0002 s pour les heuristiques et 0.012 pour le solveur pse. On voit bien que les 3 heuristiques ne coûtent rien vis à vis du temps calcul utilisé par le solveur pse.

test 5 Pour `pbsac5` (20 variables, 10 contraintes). La meilleure heuristique donne $\overline{F} = 6070$, la solution optimale obtenue est $x^* = (1,0,0,0,0,0,0,0,0,0,1,0,0,0,1,1,1,1,1,1,1,1)$ avec $F^* = 6120$. 71016 noeuds ont été créés (71015 si vous ne créez pas de noeud terminal). Sur ma machine $t_h \simeq 2.3\,10^{-4}s$, $t_s \simeq 0.3s$.

test 6 Pour `pbsac6` (39 variables, 5 contraintes). La meilleure heuristique donne $\overline{F} = 10295$. On obtient $F^* = 10618$. 21 537 596 noeuds ont été créés (2 en moins si vous ne créez pas de noeud terminal). Cela commence à être un peu long (environ 2mn 30 s sur ma machine). Il faut passer à la suite.

4 Résolution en utilisant le solveur gurobi

Il est difficile de comparer ce solveur pse "naïf" écrit en python avec Gurobi ; pour cela on pourra considérer que si on l'avait codé en C les temps de calcul seraient réduits d'environ un facteur 100.

Travail à réaliser et tests

Reprendre les explications du TP 1 et résoudre les problèmes (à partir du 4) en mesurant le temps calcul. Il n'y a pas de difficulté particulière (vérifier que vous obtenez la même valeur optimale pour la fonction). Pour enlever les informations sorties par le solveur, vous pouvez utiliser (cf correction des exercices 1 et 2 du TP 1) :

```
env = gp.Env(empty=True)
env.setParam('OutputFlag', 0)  # 0 pour retirer les infos du solveur, 1 pour les avoir
env.start()
# lors de la création d'un modèle gurobi utilisez l'argument optionnel nommé env
m = gp.Model(env=env)
```

Pour le `pbsac4`, en intégrant le temps de l'interface, j'obtiens environ [3] $0.0013\,s$ à comparer avec $0.012\,s$ obtenu par le solveur pse. Si on divise ce temps par 100, le solveur pse prend nettement l'avantage. C'est plus ou moins normal car gurobi utilise des méthodes plus sophistiquées avec un pré-solveur (simplification des équations qui parfois donne directement la solution), des heuristiques (plus sophistiquées que celles que nous avons utilisées) puis ensuite une méthode de coupe. Il y a donc un surcoût à cela pour des "petits problèmes" (exactement comme un tri insertion bat un tri rapide pour des tableaux suffisamment petits).

Pour les problèmes suivants vous allez voir que gurobi ne met pas tellement plus de temps !

3. Sur ma machine, il faut rajouter environ $0.03\,s$ vraisemblablement pour le chargement de la bibliothèque dynamique gurobi ; si on traite un autre problème à la suite, le temps calcul n'est plus impacté par ce temps de chargement.